Programación con SQL

Práctica 14

Prof. Ing. Angel Brito Segura

1. Objetivos

- Comprender los principales conceptos de programación PL/pgSQL.
- Realizar la construcción de scripts SQL empleando estructuras básicas de PL/pgSQL.
- Aplicar disparadores para resguardar la integridad de la información.

2. Conexión a servidor de PostgreSQL

Recordar que se cuenta con los siguientes ambientes en los que se ha trabajado durante el transcurso del semestre para acceder a un DBMS de PostgreSQL:

```
$ psql -h 132.248.59.8 -p 5432 -U lbd09_2027_1_<Número de cuenta> \
  -d lbd09_2027_1_<Número de cuenta>
```

Código fuente 1: Conexión al servidor PostgreSQL del laboratorio de la Facultad de Ingeniería

```
$ psql -d lab_bd_09_<num_lista>
```

Código fuente 2: Conexión local a base de datos en instalación nativa o máquina virtual

```
$ psql -h 0.0.0.0 -p 5432 -U <nombre_usuario_administrador> -d lab_bd_09_<num_lista>
```

Código fuente 3: Conexión local a base de datos en contenedor

3. Características del Lenguaje Estructurado de Consulta (SQL)

- Lenguaje de alto nivel, de estructura sencilla y fácil de entender.
- Portable por la existencia de estándares regulados.
- No procedimental.

Debido a esta última característica se cuenta con extensiones que sirven para cubrir funcionalidades no incluidas de forma directa en el estándar como son: estructuras de control (ciclos, condicionales), manejo de variables, excepciones, programación orientada a objetos, entre otras. Actualmente estas extensiones se incluyen en el estándar ISO/IEC 9075-4 `Database languages SQL Part 4: Persistent Stored Modules (SQL/PSM)`.

4. Módulos Almacenados Persistentes (PSM)

Bloques de código que contiene instrucciones SQL estándar y extensiones de procedimientos que se almacenan y ejecutan en el servidor RDBMS. Estos módulos representan alguna lógica empresarial que puede ser encapsulada, almacenada y compartida entre múltiples usuarios de bases de datos. Cada manejador define sus propias extensiones, en el caso de PostgreSQL se realiza mediante el lenguaje procedimental PL/pgSQL.

4.1. Lenguajes procedimentales de PostgreSQL

Lenguajes que en este manejador, además de SQL y C, permite la creación y uso de funciones definidas por el programador. Actualmente, en cualquier distribución estándar de PostgreSQL, se cuenta cargado por defecto los siguientes lenguajes procedimentales:

1. PL/pgSQL
2. PL/Tcl
3. PL/Perl
4. PL/Python

Existen otros lenguajes procedimentales disponibles que son desarrollados y mantenidos fuera del núcleo de una distribución de PostgreSQL. Sin importar si son adicionales o no al núcleo de distribución de este RDBMS, hay algunos lenguajes que a pesar de tener un estatus productivo se consideran como lenguajes no confiables (*untrusted*).

4.1.1. PL/pgSQL

Los objetivos al diseñarlo fueron crear un lenguaje procedimental cargable que pudiera:

- ser usado para crear funciones, procedimientos y disparadores,
- agregar estructuras de control al lenguaje SQL,
- realizar cálculos complejos,
- heredar todos los tipos, funciones, procedimientos y operadores definidos por el programador,
- ser definido para que el servidor confíe en él,
- ser fácil de usar.

Las funciones creadas con este lenguaje de programación pueden ser usadas como las construidas en el núcleo de la distribución de PostgreSQL. A partir de la versión 9.0 de este manejador, se instala de forma predeterminada.

5. Introducción a la programación PL/pgSQL

Con este lenguaje de programación podemos agrupar bloques de código y una serie de consultas del servidor de base de datos, teniendo así el poder de un lenguaje procedimental y la facilidad de uso de SQL, pero con un ahorro considerable en la sobrecarga de la red en la comunicación cliente/servidor.

5.1. Elementos de un bloque PL/pgSQL

PL/pgSQL es un lenguaje estructurado en bloques, cada bloque tiene la siguiente estructura:

```
[ << etiqueta >> ]
[DECLARE
    -- declaración de variables
]
BEGIN
    <instrucciones PL/pgSQL o SQL>
[EXCEPTION
  WHEN <nombre_condición_excepción> [OR <nombre_condición_excepción>[, ...]] THEN
    -- sentencias para manejo de la excepción
  [...]
END [ etiqueta ];
```

Código fuente 4: Bloque PL/pgSQL

Dentro de un bloque PL/pgSQL cada declaración y sentencia dentro del bloque debe terminar con un punto y coma (;).

Existen tres tipos de bloques que debe ser invocados por el programador:

- Bloques anónimos
- Procedimientos almacenados
- Funciones

5.2. Bloques anónimos

Este tipo de bloques no contienen un nombre y no son almacenados en la base de datos, debido a esto, se compilan y se ejecutan cada vez que se requieran (deben de estar embebidos en una aplicación).

```
DO $$
  [DECLARE
    -- declaración de variables
  ]
  BEGIN
    <instrucciones PL/pgSQL o SQL>
  END;
$$;
```

Código fuente 5: Bloque anónimo PL/pgSQL

5.3. Reportar errores y mensajes

La sentencia `RAISE` nos permite reportar mensajes y generar errores.

```
RAISE [{DEBUG|LOG|INFO|NOTICE|WARNING|EXCEPTION}] '<mensaje>'[, <expresión>[,...]];
```

Código fuente 6: Sentencia RAISE

```
DO $$
  BEGIN
    RAISE NOTICE 'Hola mundo! La fecha actual es: %', current_date;
  END;
```

```
5 $$;
```

Código fuente 7: Primer programa en PL/pgSQL

5.4. Declaración de variables

Los tipos de datos que se emplean para declarar variables son los integrados en PostgreSQL y algunos específicos de este lenguaje de programación (`%TYPE`, `%ROWTYPE`). Su alcance es local, es decir, será visible únicamente en el bloque PL/SQL donde fue declarada.

```
<nombre_variable> [CONSTANT] <tipo_dato> [NOT NULL] [{DEFAULT | := | =} <valor_defecto>];
```

Código fuente 8: Declaración de variable

5.5. Cursores

Estructuras de control utilizados para realizar operaciones con los registros devueltos tras ejecutar una sentencia `SELECT`. Existen dos tipos:

- **Implícitos**: Creados y administrados internamente por el manejador para procesar sentencias SQL (devuelven un solo registro).
- **Explícitos**: Declarados explícitamente por el programador.

```
<nombre_cursor> CURSOR [(<argumentos>)] FOR <consulta-sql>;
```

Código fuente 9: Declaración de cursor explícito

Es necesario abrir el cursor explícito mediante la siguiente sintaxis:

```
OPEN <nombre_cursor> [([<nombre_argumento> :=]<valor>[, ...])];
```

Código fuente 10: Abrir cursor explícito creado

Para recuperar la siguiente fila del cursor se cuenta con la siguiente sentencia:

```
FETCH [<dirección> {FROM | IN} ] <nombre_cursor> INTO <nombre_variable>;
```

Código fuente 11: Extracción de datos del cursor explícito creado

Es muy importante que al terminar se cierre el cursor:

```
CLOSE <nombre_cursor>;
```

Código fuente 12: Cierre del cursor explícito creado

Un cursor define atributos que se emplean para controlar el acceso a los datos, de los cuales destacan:

- `%FOUND`: Indica si la última operación FETCH devolvió una fila.
- `%NOTFOUND`: Indica si la última operación FETCH no devolvió una fila.
- `%ISOPEN`: Indica si el cursor está abierto.
- `%ROWCOUNT`: Devuelve el número de filas extraídas por el cursor.

Para simplificar el manejo de cursores, se cuenta con una variante de la estructura de control `FOR`:

```
[<< <etiqueta> >>]
FOR <nombre_variable> IN <nombre_cursor> [([<nombre_argumento> :=]<valor>[, ...])]] LOOP
    -- sentencias para tratar la información
END LOOP [<etiqueta>];
```

Código fuente 13: Iteración de cursor explícito creado

Tener mucho cuidado si el cursor se manda a llamar desde fuera del RBDMS ya que se pudieran presentar problemas de rendimiento al recuperar una fila del cursor por el tiempo necesario para enviar la petición al servidor de la base de datos y esperar su respuesta.

5.6. Funciones

Son objetos con nombre almacenados en la base de datos que no requieren ser recompilados cada vez que se deseen ejecutar. Deben de regresar un valor por lo que puede ser invocada desde una sentencia SQL, así como ser el operando derecho en una expresión (son iguales a otra función definida en el núcleo de la distribución de PostgreSQL). Como se puede observar en el siguiente código se utilizan comillas con el signo de $ para escribir el cuerpo de la función y con esto poder utilizar comillas simples o barras invertidas en las instrucciones del bloque `BEGIN`:

```
1  CREATE [OR REPLACE] FUNCTION [<nombre_esquema>.]<nombre_función>(
2    [IN|OUT|INOUT|VARIADIC] [<nombre_argumento>] <tipo_dato> [{DEFAULT|=} <valor_defecto>]
3    [, ...]
4  )
5  RETURNS <tipo_dato> AS $[<etiqueta>]$
6  [DECLARE
7      -- declaración de variables locales
8  ]
9  BEGIN
10     -- cuerpo de la función con instrucciones SQL y PL/pgSQL
11   RETURN <expresión/valor>;
12 END;
13 $[<etiqueta>]$ LANGUAGE plpgsql;
```

Código fuente 14: Definición de función en PL/pgSQL

Para poder definir una función, el programador/usuario debe contar con el privilegio `USAGE` para este lenguaje:

```
GRANT USAGE ON LANGUAGE plpgsql TO <nombre_usuario_bd>;
```

Código fuente 15: Privilegio para definir funciones en PostgreSQL

Para conocer más información, se puede consultar la documentación oficial de la sentencia CREATE FUNCTION.

5.7. Procedimientos almacenados

Programa que es compilado y almacenado físicamente en el servidor de la base de datos. Una vez realizado esto, no se requiere volver a escribir todas las instrucciones sino únicamente hacer referencia al procedimiento. Esto mejora el rendimiento del servidor, ya que el programa se compila una sola vez y se ejecuta N veces.

A diferencia de los bloques anónimos, un procedimiento requiere de un nombre y son similares a una función pero éste no retorna valor. A partir de la versión 11 de PostgreSQL se cuenta con soporte nativo.

```
1  CREATE [OR REPLACE] PROCEDURE [<nombre_esquema>.]<nombre_procedimiento>(
2    [IN|OUT|INOUT|VARIADIC] [<nombre_argumento>] <tipo_dato> [{DEFAULT|=} <valor_defecto>]
3    [, ...]
```

```
)
AS $[<etiqueta>]$
[DECLARE
    -- declaración de variables locales
]
BEGIN
    -- cuerpo del procedimiento con instrucciones SQL y PL/pgSQL
END;
$[<etiqueta>]$ LANGUAGE plpgsql;
```

Código fuente 16: Creación de procedimiento almacenado en PL/pgSQL

Para conocer más información, se puede consultar la documentación oficial de la sentencia CREATE PROCEDURE.

```
CALL [<nombre_esquema>.]<nombre_procedimiento>([<valor_argumento1> [, ...]]);
```

Código fuente 17: Ejecución de procedimiento almacenado

Para conocer más información, se puede consultar la documentación oficial de la sentencia CALL.

5.8. Disparador (*Trigger*)

Código procedimental que es automáticamente invocado por el RDBMS dada la ocurrencia de algún evento de manipulación de datos (sentencias DML). Su principal diferencia con respecto a otros programas PL/pgSQL como procedimientos o funciones, es que no puede ser invocado por el programador.

```
CREATE [OR REPLACE] [CONSTRAINT] TRIGGER [<nombre_esquema>.]<nombre_disparador>
{BEFORE | AFTER | INSTEAD OF}
{INSERT | UPDATE [OF <nombre_columna> [, ... ]] | DELETE | TRUNCATE [OR ...] }
  ON [<nombre_esquema>.]<nombre_tabla>
    [FOR [EACH] {ROW | STATEMENT}]
    [WHEN (<condición>)]
    EXECUTE {FUNCTION | PROCEDURE} <nombre_función>([<argumentos>]);
```

Código fuente 18: Definición de función en PL/pgSQL

Para conocer más información, se puede consultar la documentación oficial de la sentencia CREATE TRIGGER.

6. Caso de estudio

Del modelo relacional del sistema de inscripciones a ALLUNAM:

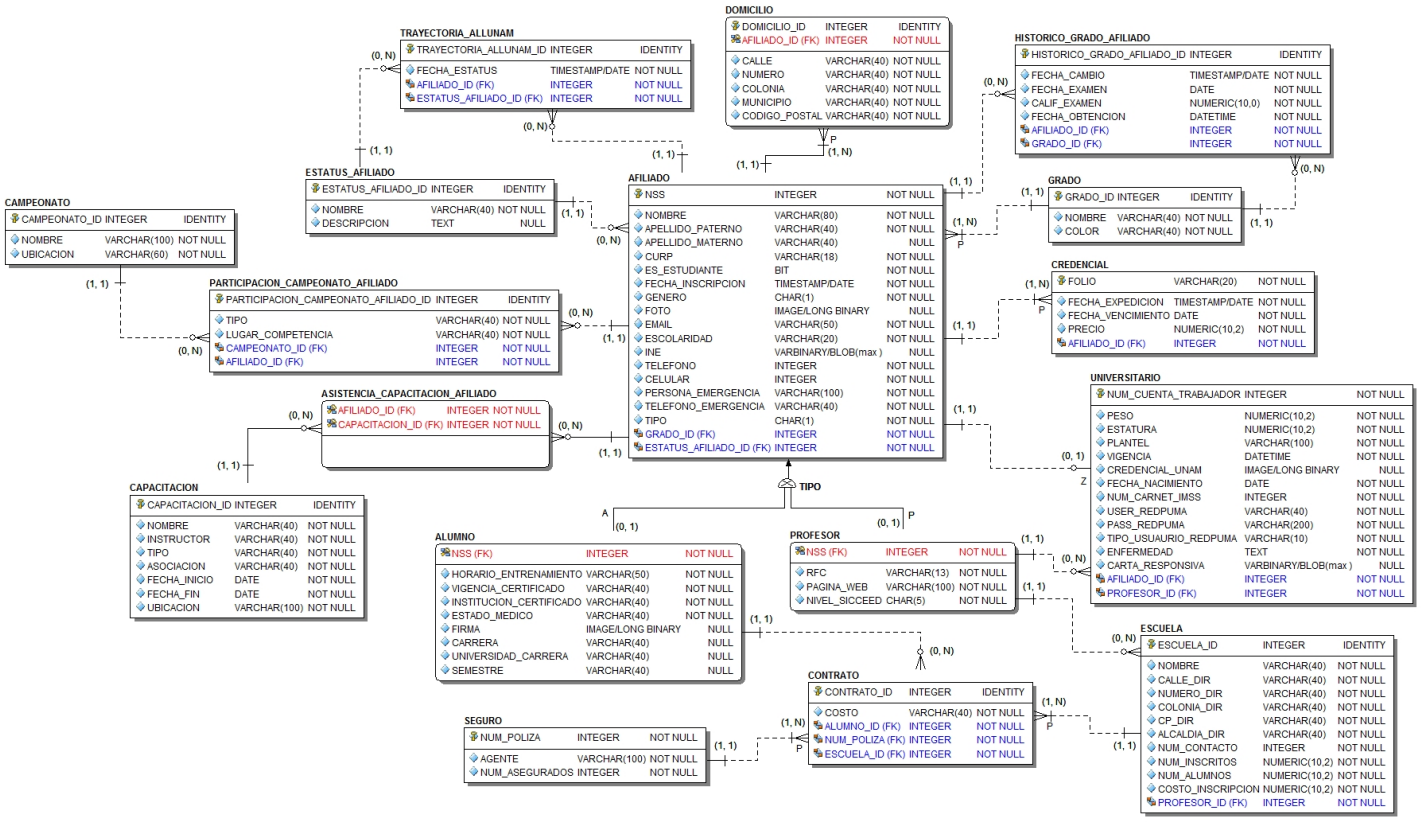


Figura 1: Sistema de Inscripciones a la ALLUNAM

6.1. Disparador para administrar el estatus de un afiliado

Crear un disparador `t_estatus_afiliado-<número_cuenta>` que ayude a validar lo siguiente:

- Al registrar un nuevo afiliado el estatus siempre deberá ser DEPORTISTA.
- Cuando se registren más de 3 asistencias a capacitaciones, debe cambiar a JUEZ.
- Cuando el estatus del afiliado cambie a ALTO RENDIMIENTO, se deberá verificar que existan por lo menos 3 participaciones en campeonatos teniendo como lugar mínimo obtenido el tercero.
- Sólo los afiliados de grados intermedios o superiores pueden tener estatus de SELECTIVO.

A) Crear un script `s-01-trigger-check-status.sql` que contenga la definición del disparador.

B) Crear un script `s-02-trigger-check-status-test.sql` que contenga a uno o más bloques anónimos empleados para validar el correcto funcionamiento del disparador.

6.2. Procedimiento almacenado para registro de profesores

Crear un procedimiento almacenado `sp_registra_profesores-<número_cuenta>` que registre los datos de un profesor y lo asocie a una escuela introducida. Al registrarse el profesor, se debe de validar que se cambie el tipo de afiliado. Sólo se podrán registrar aquellos que tengan grados de color café o superiores y no hayan reprobado ningún examen (calificación abajo de 6). Como parámetros recibirá nss del afiliado a registrar como profesor, rfc, página web (debe iniciar con el protocolo) y nivel en el SiCCEED (sólo se aceptan con nomenclatura N<número>).

a) Crear un script `s-03-sp-new-professor.sql` que contenga la definición del procedimiento.

b) Crear un script `s-04-sp-new-professor-test.sql` que contenga a uno o más bloques anónimos empleados para probar el correcto funcionamiento del procedimiento.

Referencias

[1] The PostgreSQL Global Development Group. «Procedural Languages. »dirección: https://www.postgresql.org/docs/15/xplang.html (accedido: 20.04.2024).

[2] J. Rodríguez, «Introducción a la programación,» en *Bases de Datos*, Facultad de Ingeniería, UNAM, ed., 2024.

[3] The PostgreSQL Global Development Group. «PL/pgSQL - SQL Procedural Language. »dirección: https://www.postgresql.org/docs/15/plpgsql.html (accedido: 20.04.2024).

[4] L. Czeslaw, «Introducción a la programación con SQL,» en *Bases de Datos*, Facultad de Ingeniería, UNAM, ed., 2020.

[5] The PostgreSQL Global Development Group. «External Projects: Procedural Languages. »dirección: https://www.postgresql.org/docs/15/external-pl.html (accedido: 20.04.2024).

[6] L. Arellano, L. Hernández y J. Rodríguez, *Manual de prácticas del Laboratorio de Bases de Datos*. UNAM, Facultad de Ingeniería, DIE, 2018.

tiene licencia de Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International.